AI + CTF，我是如何用本地大模型拿下山东省赛冠军的？解题数：17/20，7个一血

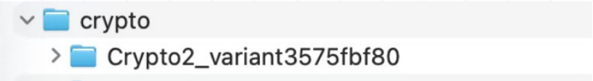
原创 Real返璞归真 Real返璞归真

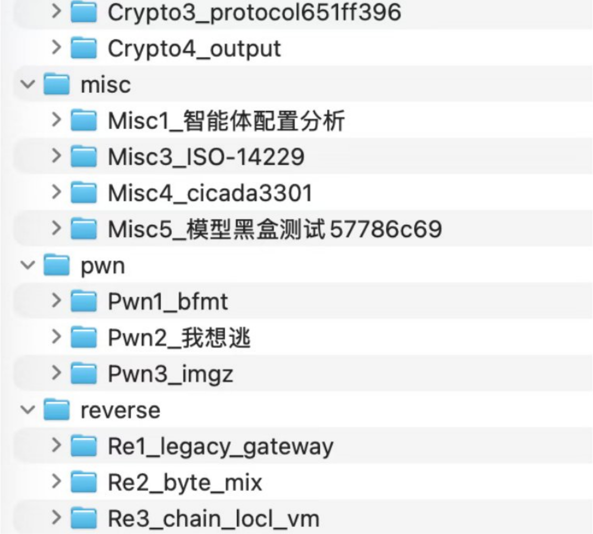
2026年10月26日 09:35 山东 6人 AI摘要

前言

公众号

关注公众号【Real返璞归真】回复【山东省赛2026】获取全部题目附件下载地址：



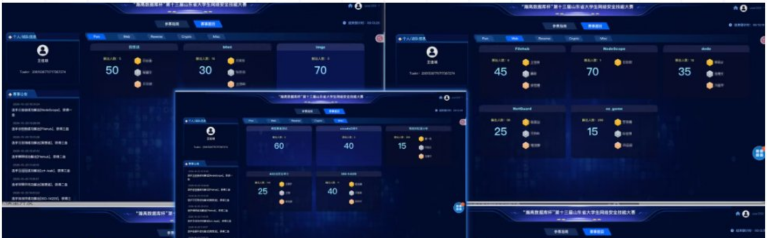


Crypto3_protocol651ff396
Crypto4_output
misc
Misc1_智能体配置分析
Misc3_ISO-14229
Misc4_cicada3301
Misc5_模型黑盒测试57786c69
pwn
Pwn1_bfmt
Pwn2_我想逃
Pwn3_imgz
reverse
Re1_legacy_gateway
Re2_byte_mix
Re3_chain_locl_vm

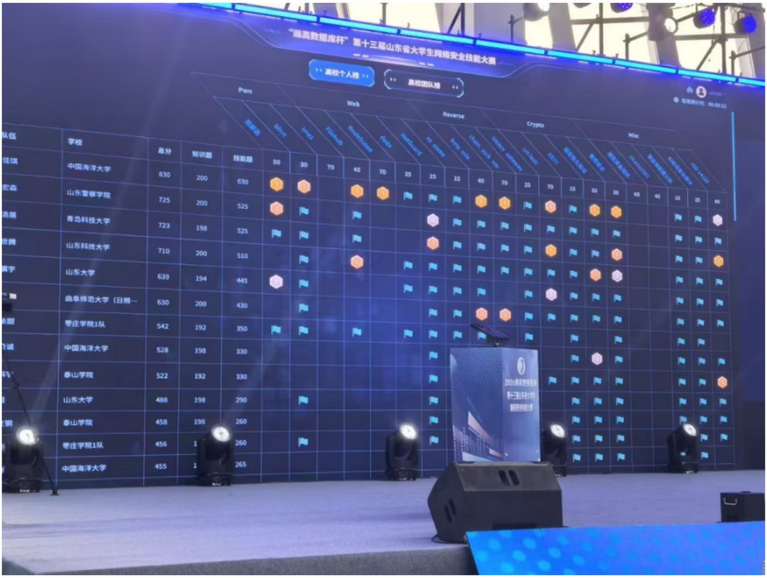
•• AI摘要·35人

解题情况

比赛结束前10分钟各赛题解题情况：



40
70
25
70
15
30
30



队伍	学校	总分	知识题	技能题
佳琪	中国海洋大学	830	200	630
宏淼	山东警察学院	725	200	525
浩展	青岛科技大学	723	198	525
世腾	山东科技大学	710	200	510
展宇	山东大学	639	194	445
	曲阜师范大学（日照…	630	200	430
欣甜	枣庄学院1队	542	192	350
飞诚	中国海洋大学	528	198	330
	泰山学院	522	192	330
	山东大学	488	198	290
	泰山学院	458	198	260
	枣庄学院1队	456	1	260
	中国海洋大学	455	1	265

赛况简介

第十三届山东省网络安全技能大赛（瀚高数据库杯）于10月25日在枣庄学院举办。

鉴于往年的比赛情况，今年组委会在反作弊方面下了很大功夫：

- 每台电脑使用USB锁封锁多余的接口
- 每个已使用的USB接口使用一次性封条贴住
- 厕所由组委会工作人员严格控制出入
- 每个桌子有一名志愿者负责监考

为了防止黑幕，比赛前我开启了录屏+录像。虽然今年解题很快，但是没有和去年（）一样的志愿者全程盯着我的电脑、搜查物品和汇报我的解题进度。

本次比赛中，我使用AI辅助解题，最终解题数：17/20（包含7个1血、1个2血、2个3血）。

备赛阶段，学习到了很多关于LLM的知识，也深入了解一些LLM的薄弱点与能力边界。

LLM辅助安全，提高效率，但安全研判的主体依旧是人。

对于在CTF中使用Agent辅助解题，并不是简单的有合适的硬件、装上模型就可以，还需要对LLM和

Agent的工作原理充分了解、选取合适的模型框架与参数、大量的赛前准备以及赛中的人机协同。

本篇文章主要分享一下个人关于 AI辅助CTF 的一些经验摸索，供各位师傅参考。

AI辅助解题

简介

今年的比赛相比于往年发生了一个很大的变化：在AI Agent时代，很多选手开始使用**AI辅助解题**。

通过今年的排行榜，可以发现几个明显的特征：

- Crypto方向（AI最擅长的数学推理领域）解题数增多，有6名师傅AK。
- 理论题（AI知识丰富）均分增高，且有数名师傅满分。
- 选手们普遍解题速度更快，解题数量更多。

赛前，我也准备了很久本地AI环境，并探索各种场景

下的最优解。

先揭晓一下答案，经过长时间的摸索和反复的bench测试，最终我采用的解决方案（Mac Silicon架构）：oMLX + Qwen3.8-Flash-Next + Pi Agent + 提示词 + 外部工具

最初的模型速度很慢、解题能力也比较弱，但是经过一些优化可以明显提高能力。

最终在4并发下，总体吞吐量达到 120 token / s，模型占用内存80G。在262k上下文限制下，每个会话占用约5-10G KV Cache。总占用105G左右。

（Tips：我的电脑配置是MacBook M5 Max 128G，融合内存带宽614GB / s，师傅们可以参照文章并结合自己的机器去配置调试，据说Strata已经能在32G显存下跑Qwen3.8-Flash-Next了）

需要关注的问题

大概需要关注几个问题：

- 内存：决定本机能运行什么参数量的模型
- 带宽：决定模型的速度（token/s），很容易被忽视的地方
- Agent：选取合适的Agent搭配模型实现自动化解题
- LLM：选取什么模型？考虑参数量与量化程度。
- 推理框架：选取什么推理框架运行模型？不同推理框架可能导致速度产生很大的差异。
- 提示词 / Skills / 知识库：是否需要为Agent增加提示词 / Skills /知识库？

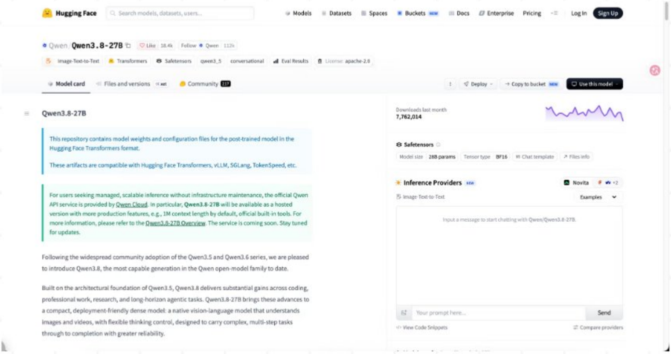
下面，我们将从 **模型选取、推理框架选取、模型量化、提示词 / Skills、并发、外部工具框架** 等方面，分享我是如何用AI解题的。

1.模型选取

目前，比较优秀的模型：

- Qwen3.8-27B （https://huggingface.co/Qwen/Qwen3.8-

27B）



Qwen / **Qwen3.8-27B** Like 18.4k Follow Qwen 112k

Image-Text-to-Text · Transformers · Safetensors · qwen3_5 · conversational · Eval Results · License: apache-2.0

Community 217

Qwen3.8-27B

This repository contains model weights and configuration files for the post-trained model in the Hugging Face Transformers format.

These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang, TokenSpeed, etc.

For users seeking managed, scalable inference without infrastructure maintenance, the official Qwen API service is provided by Qwen Cloud. In particular, **Qwen3.8-27B** will be available as a hosted version with more production features, e.g., 1M context length by default, official built-in tools. For more information, please refer to the Qwen3.8-27B Overview. The service is coming soon. Stay tuned for updates.

Following the widespread community adoption of the Qwen3.5 and Qwen3.6 series, we are pleased to introduce Qwen3.8, the most capable generation in the Qwen open-model family to date.

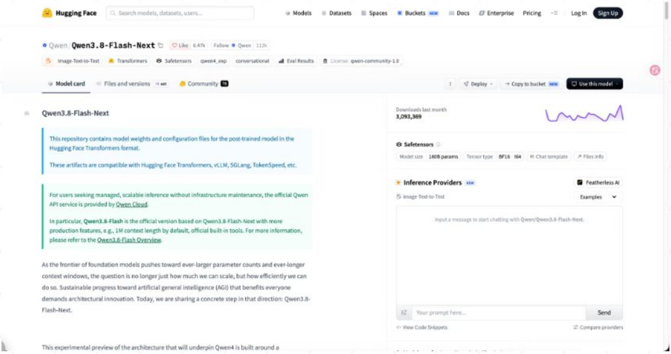
Built on the architectural foundation of Qwen3.5, Qwen3.8 delivers substantial gains across coding, professional work, research, and long-horizon agentic tasks. Qwen3.8-27B brings these advances to a compact, deployment-friendly dense model: a native vision-language model that understands images and videos, with flexible thinking control, designed to carry complex, multi-step tasks through to completion with greater reliability.

Downloads last month
7,762,014

Safetensors
Model size 28B params · Tensor type BF16 · Chat template · Files info

Inference Providers: Novita +2
Image-Text-to-Text
Input a message to start chatting with Qwen/Qwen3.8-27B.
Your prompt here...

  - Qwen3.8-Flash-Next（https://huggingface.co/Qwen/Qwen3.8-Flash-Next）



Qwen / **Qwen3.8-Flash-Next** Like 6.47k Follow Qwen 112k

Image-Text-to-Text · Transformers · Safetensors · qwen4_exp · conversational · Eval Results · License: qwen-community-1.0

Qwen3.8-Flash-Next

This repository contains model weights and configuration files for the post-trained model in the Hugging Face Transformers format.

These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang, TokenSpeed, etc.

For users seeking managed, scalable inference without infrastructure maintenance, the official Qwen API service is provided by Qwen Cloud.

In particular, **Qwen3.8-Flash** is the official version based on Qwen3.8-Flash-Next with more production features, e.g., 1M context length by default, official built-in tools. For more information, please refer to the Qwen3.8-Flash Overview.

As the frontier of foundation models pushes toward ever-larger parameter counts and ever-longer context windows, the question is no longer just how much we can scale, but how efficiently we can do so. Sustainable progress toward artificial general intelligence (AGI) that benefits everyone demands architectural innovation. Today, we are sharing a concrete step in that direction: Qwen3.8-Flash-Next.

This experimental preview of the architecture that will underpin Qwen4 is built around a

Downloads last month
3,093,369

Safetensors
Model size 180B params · Tensor type BF16 · I64 · Chat template · Files info

Inference Providers: Featherless AI
Image-Text-to-Text
Input a message to start chatting with Qwen/Qwen3.8-Flash-Next.
Your prompt here...

其中：

- `Qwen3.8-27B` 是一个参数量为27b（270亿）参数的**稠密（Dense）模型**

- `Qwen3.8-Flash-Next` 是**Moe（混合专家）模型**，包含125B参数和51B的N-gram，并且它是 `Qwen4` 新架构的预览版。

通俗易懂的来说，**稠密模型**每个 `token` 都会激活全部的27b参数，**Moe模型**每个 `token` 实际参与计算的只有6b参数。

也就是说， `Qwen3.8-27B` 更擅长强推理的场景，而 `Qwen3.8-Flash-Next` 有更多的知识、更快的速度。

对于CTF解题赛来说，显然 `Qwen3.8-Flash-Next` 是更好的选择，它有更丰富的安全知识，并且可以快速交互。

快速交互非常关键，对于除Crypto方向外的其它赛题，AI并不需要太强的推理（思考）能力，而是需要大量的工具交互信息。

顺带提一下：这两个模型都支持Thinking能力，也都具备视觉能力。

如果电脑GPU的显存比较小，可参考 8.拓展阅读 部分对模型分层加载，目前比较成熟的方案：

- Strata （Windows和Mac通用，显存、内存和SSD分层加载）
- oMLX（Mac专用，专家分层加载）

并且，他们都会将51B的N-gram表放到磁盘。

2.推理框架

推理框架用于：加载模型、调度计算资源、管理显存，并高效地生成回答。

对于开源模型，官方默认发布 F16 safetensors 格式的模型参数文件，主要面向 `vLLM` 和 `SGLang` 推理框架，这两个框架主要用于生产级的高并发服务引擎，需要搭配专业的服务器GPU。

而消费级产品（游戏本、MacBook）CPU通常使

用 `llama.cpp` 和 `MLX` 框架部署模型。

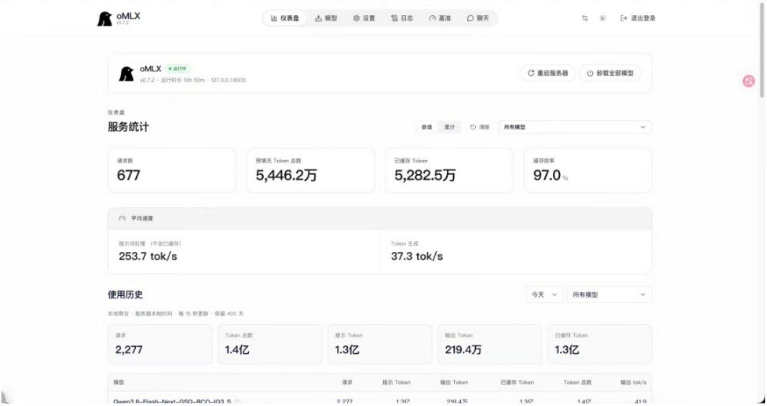
模型参数文件开源后，陆续会有社区版模型出现，它们会将官方格式转换为适用于消费级框架部署的格式。如：适用于 `llama.cpp` 的 **GGUF** 格式文件 和 适用于 **MLX** 的文件。

如何安装推理框架呢？有几个可供选择的方式（我逐个尝试了）：

- `llama.cpp` ：直接安装 `llama.cpp` 框架，纯命令操作，但是可以配置很多自定义参数，如：后面要提到的**MTP加速**。
- `ollama` ：基于 `llama.cpp` 的可视化界面客户端，操作简单，但是很多设置不能自定义（不知道能不能通过命令行定义）。
- `oMLX` ：针对Mac的**Silicon**架构设计，在MacBook上可以获得更高的tps(token/s)速度。
- `LM Studio` ：避雷，适合傻瓜式安装操作，启动简单。但很多设置不能自定义。

总的来说，如果你是Windows建议安装 `llama.cp`

`p`，如果你是Mac建议按照 `oMLX` 。



oMLX 运行中
v0.7.2 · 运行时长 10h 50m · 127.0.0.1:8000

仪表盘
服务统计

请求数 677
预填充 Token 总数 5,446.2万
已缓存 Token 5,282.5万
缓存效率 97.0%

平均速度
提示词处理（不含已缓存） 253.7 tok/s
Token 生成 37.3 tok/s

使用历史
本地聚合 · 服务器本地时间 · 每 15 秒更新 · 保留 400 天

请求 2,277
Token 总数 1.4亿
提示 Token 1.3亿
输出 Token 219.4万
已缓存 Token 1.3亿

模型	请求	提示 Token	输出 Token	已缓存 Token	Token 总数	输出 tok/s
Qwen3.8-Flash-Next-GSQ-BCQ-IQ3_S	2,277	1.3亿	219.4万	1.3亿	1.4亿	41.9

除了格式转换适配，社区版模型也会去做**量化**，这也是我们接下来要说的。

3.模型量化

官方开源的模型是满精度模型，常以 **BF16（Brain Floating Point 16）** 格式发布。

通俗易懂的来说，也就是每个参数占16bit（2字节）。

`量化` 就是将参数的浮点数精度压缩，常见的量化：

8bit、6bit、4bit、甚至2bit。

例如：4-bit量化就是以4bit浮点数存储原来16bit浮点数。

`Qwen3.8-27B` 在 BF16 下大概需要56G显存，而4bit量化只需要16G显存。

通过模型量化（参数精度压缩），可以让低显存的设备跑更高参数的模型、得到更快的速度，但随之而来的是模型能力下降。

通过我的实测，发现4-bit和8-bit在做CTF题目的质量差距不大，但是4-bit的速度远快于8bit的速度（2倍）。

因此，我本地选取了4-bit量化版本。

4.模型调优

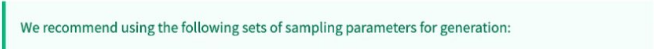
安装完推理框架、下载完量化版的模型权重文件后，就可以运行模型了。

`oMLX` 自带bench功能，可以测试模型输出速度，起

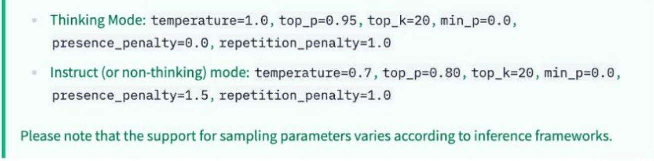
初我只有30 token/s。

对于简单的模型调优来说，主要是两点：

1. 按照官方文档配置 `temperature`、`top_p`、`top_k`、`min_p` 等参数：



We recommend using the following sets of sampling parameters for generation:

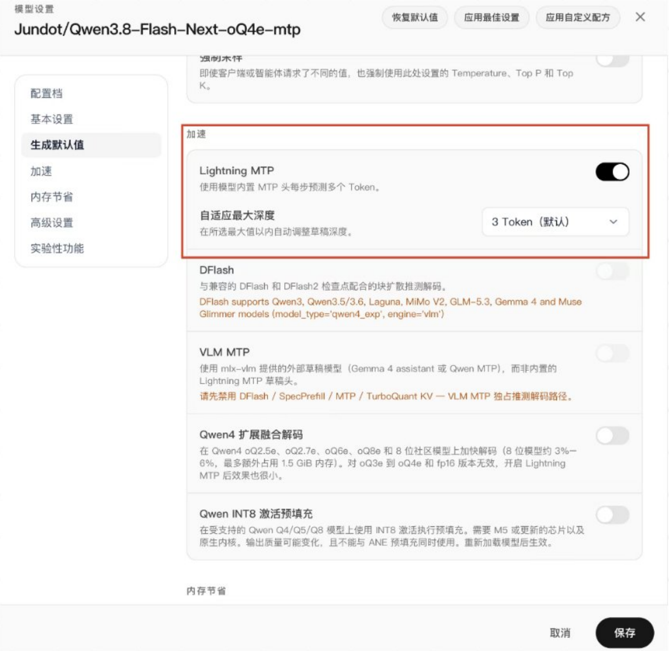


- Thinking Mode: `temperature=1.0`, `top_p=0.95`, `top_k=20`, `min_p=0.0`, `presence_penalty=0.0`, `repetition_penalty=1.0`
- Instruct (or non-thinking) mode: `temperature=0.7`, `top_p=0.80`, `top_k=20`, `min_p=0.0`, `presence_penalty=1.5`, `repetition_penalty=1.0`

Please note that the support for sampling parameters varies according to inference frameworks.

这一步很重要，最初我没有按照官方文档去配置参数，发现理论题和CTF题输出质量下降很严重。

2. 打开MTP加速：



Jundot/Qwen3.8-Flash-Next-oQ4e-mtp

即使客户端或智能体请求了不同的值，也强制使用此处设置的 Temperature、Top P 和 Top K。

Lightning MTP ●
使用模型内置 MTP 头每步预测多个 Token。

自适应最大深度：3 Token（默认）
在所选最大值以内自动调整草稿深度。

DFlash ○
与兼容的 DFlash 和 DFlash2 检查点配合的块扩散推测解码。
DFlash supports Qwen3, Qwen3.5/3.6, Laguna, MiMo V2, GLM-5.3, Gemma 4 and Muse Glimmer models (model_type='qwen4_exp', engine='vlm')

VLM MTP ○
使用 mlx-vlm 提供的外部草稿模型（Gemma 4 assistant 或 Qwen MTP），而非内置的 Lightning MTP 草稿头。
请先禁用 DFlash / SpecPrefill / MTP / TurboQuant KV — VLM MTP 独占推测解码路径。

Qwen4 扩展融合解码 ○
在 Qwen4 oQ2.5e、oQ2.7e、oQ6e、oQ8e 和 8 位社区模型上加快解码（8 位模型约 3%–6%，最多额外占用 1.5 GiB 内存）。对 oQ3e 到 oQ4e 和 fp16 版本无效，开启 Lightning MTP 后效果也很小。

Qwen INT8 激活预填充 ○
在受支持的 Qwen Q4/Q5/Q8 模型上使用 INT8 激活执行预填充。需要 M5 或更新的芯片以及原生内核。输出质量可能变化，且不能与 ANE 预填充同时使用。重新加载模型后生效。

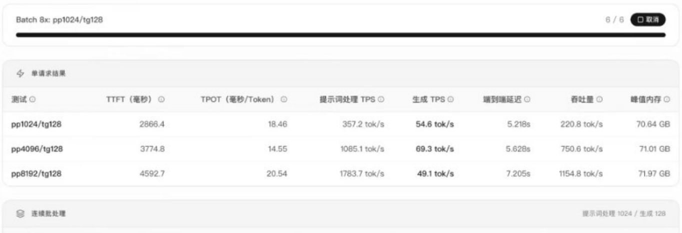
`Qwen` 的两个模型都内置了MTP头，用于MTP加速。MTP加速的大概原理是批量猜测后续token，然后进行验证，如果猜测成功，就可以节省后续前向传播的次数。

这里我也尝试了其它各种加速方式，包括 外部MTP模型、DFlash 等，发现内置MTP加速的效果最好。

开启MTP加速后bench可以到达60 token/s（速度翻倍）。

5.模型并发

bench实测：

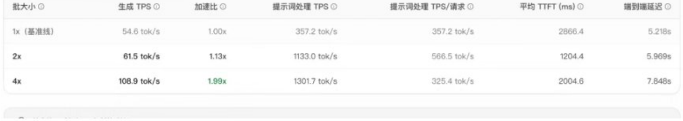

Batch 8x: pp1024/tg128

单请求结果

测试	TTFT（毫秒）	TPOT（毫秒/Token）	提示词处理 TPS	生成 TPS	端到端延迟	吞吐量	峰值内存
pp1024/tg128	2866.4	18.46	357.2 tok/s	**54.6 tok/s**	5.218s	220.8 tok/s	70.64 GB
pp4096/tg128	3774.8	14.55	1085.1 tok/s	**69.3 tok/s**	5.628s	750.6 tok/s	71.01 GB
pp8192/tg128	4592.7	20.54	1783.7 tok/s	**49.1 tok/s**	7.205s	1154.8 tok/s	71.97 GB

连续批处理

提示词处理 1024 / 生成 128



批大小	生成 TPS	加速比	提示词处理 TPS	提示词处理 TPS/请求	平均 TTFT (ms)	端到端延迟
1x（基准线）	54.6 tok/s	1.00x	357.2 tok/s	357.2 tok/s	2866.4	5.218s
2x	61.5 tok/s	1.13x	1133.0 tok/s	566.5 tok/s	1204.4	5.969s
4x	108.9 tok/s	1.99x	1301.7 tok/s	325.4 tok/s	2004.6	7.848s

通俗易懂的说，并发调用LLM比单独调用LLM的利用率更高。

单并发下，经过优化达到 55 token / s。四并发下，108.9 token / s。

虽然每个会话只有26 token / s，但是每秒的token量增加，对于总体来说是收益。

也就是说，整体单位时间处理的请求数（QPS/吞吐量）会提升。

这些是一些简单的优化，关于kv cache缓存等，或许也有一些优化方案，师傅们也可以自行去探索。

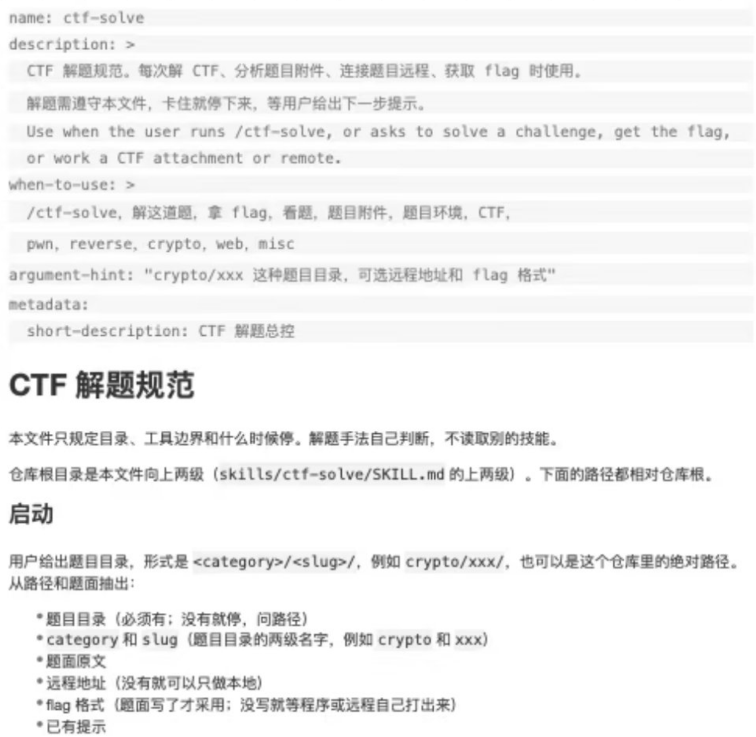
6.提示词 / skills

比较热门的CTF Skills：

https://github.com/ljagiello/ctf-skills

但是经过尝试，我发现对于本地小模型。skills会降低首字的速度，并误导模型严格按照skills的方式去执行。

因此，在比赛中，我没有使用这个庞大的skills，而是写了一个简短的提示词：



```
name: ctf-solve
description: >
  CTF 解题规范。每次解 CTF、分析题目附件、连接题目远程、获取 flag 时使用。
  解题需遵守本文件，卡住就停下来，等用户给出下一步提示。
  Use when the user runs /ctf-solve, or asks to solve a challenge, get the flag,
  or work a CTF attachment or remote.
when-to-use: >
  /ctf-solve，解这道题，拿 flag，看题，题目附件，题目环境，CTF，
  pwn，reverse，crypto，web，misc
argument-hint: "crypto/xxx 这种题目目录，可选远程地址和 flag 格式"
metadata:
  short-description: CTF 解题总控
```

CTF 解题规范

本文件只规定目录、工具边界和什么时候停。解题手法自己判断，不读取别的技能。

仓库根目录是本文件向上两级（`skills/ctf-solve/SKILL.md` 的上两级）。下面的路径都相对仓库根。

启动

用户给出题目目录，形式是 `<category>/<slug>/`，例如 `crypto/xxx/`，也可以是这个仓库里的绝对路径。从路径和题面抽出：

- 题目目录（必须有；没有就停，问路径）
- `category` 和 `slug`（题目目录的两级名字，例如 `crypto` 和 `xxx`）
- 题面原文
- 远程地址（没有就可以只做本地）
- flag 格式（题面写了才采用；没写就等程序或远程自己打出来）
- 已有提示

一次只解一道。多道题时，每题用自己的工作目录，做完或把卡点写入状态后再进下一道。

工作目录

先拷贝，再解题。题目目录保持原样，之后不再写入。

工作副本是：

```
workspace/<category>/<slug>/
```

题目在 `crypto/xxx/` 时，副本就是 `workspace/crypto/xxx/`。

工作目录还不存在时，整份拷过去：

```
mkdir -p workspace/<category>
cp -a <category>/<slug> workspace/<category>/
```

然后所有阅读、运行、修改都在工作副本里做。

工作目录已经存在：这是续作。不要再次拷贝，以免盖掉已有脚本和 `STATUS.md`。先读 `workspace/<category>/<slug>/STATUS.md`，从 `next` 继续，已写明失败的实验不要再跑。

拷贝完成后把状态写成：

```
category:
flag:
hypothesis:
facts:
next:
```

`facts` 一行一条已证实的事实。假设变了就改 `hypothesis` 和 `next`。拿到 flag 后把 `flag` 和复现命令写上。这是续作记录，不是 writeup。用户没要 writeup 就不写。

开场

在工作副本里做一遍，然后写 `STATUS.md`：

1. 列出文件、大小、`file` 类型。有 `Dockerfile`、`docker-compose`、说明文件、git 仓库时先读说明和最近提交。
2. 对二进制和未知文件跑一次 `strings -a -n 6`。对文本搜一次 `flag{`、`ctf{`、`FLAG{`、`key`、`password`。
3. 搜到候选 flag 时，先核对它是不是题面要求的输出。依赖库、示例 README、测试向量里的字符串不算解出。

然后进入循环。`category` 只决定工作目录，不表示还有另一份解题技能。

工具

比赛机断网。只使用 Mac、Ubuntu、Kali 和已打开的 MCP 里已经装好的工具。不要 `pip install`、`apt`、`brew`、下载安装包或克隆工具仓库。缺工具时按下面的顺序换地方；三处都没有就停下来问用户，不要尝试安装。

本机是 MacBook。能在本机用 Python 解完的题，不要进虚拟机。本机 Python 带常用库，适合这些题：

- 不需要 sage 的 crypto
- 分析之后写脚本就能解的 reverse，且不需要 x64 调试
- misc
- 只需要 Python 发网络请求的 web

需要 GUI 或专用工具时：

工具	用法

IDA-MCP	已注册为工具，直接调用
Jadx-MCP	先停下来，请用户打开。用户确认后再调用
x64dbg-MCP	先停下来，请用户打开。用户确认后再调用
BurpSuite-MCP	先停下来，请用户打开。用户确认后再调用

sage 和其他解题工具在 Ubuntu 虚拟机。命令形式：

```
ssh ubuntu 'bash -lc "命令"'
```

Ubuntu 里没有这个工具，再去 Kali 虚拟机：

```
ssh kali 'bash -lc "命令"'
```

两台都没有，或者工具在 Windows 虚拟机里，就停下来问用户，请用户在 Windows 里手动做。停下来时写清要打开的程序或要执行的操作，以及用哪个文件。把这件事写入 `STATUS.md` 的 `next`。

使用 Ubuntu 或 Kali 之前，把 Mac 上的 `workspace/<category>/<slug>/` 拷到那台虚拟机的 `~/workspace/<category>/<slug>/`。虚拟机里只在这个目录工作。远程还没有该目录时：

```
ssh ubuntu 'bash -lc "mkdir -p ~/workspace/<category>"'
rsync -a workspace/<category>/<slug>/ ubuntu:~/workspace/<category>/<slug>/
```

Kali 把主机名换成 `kali`。远程目录已经存在就接着用，不要整份覆盖。虚拟机里新生成的脚本和结果拷回 Mac 的同一工作副本，不要覆盖 Mac 上的 `STATUS.md`，也不要写回题目原目录。

循环

每一轮实验之前先写清两件事：这条实验在验证哪句假设；什么结果出现就否定它。

做当前最便宜、最能把假设分开的那一个实验。优先本地、静态、只读。本地能解完就先不连远程。

实验结果要变成一句新事实，写进 `facts`。下面三种都不算新事实：命令报错但原因没弄清、把同一条命令换个参数再跑、把工具输出原样贴进笔记。

连续三轮没有新事实，就停下来，把 `STATUS.md` 更新好，按「完成」一节汇报卡点。等用户给出下一步提示后再继续。工作目录不动。

在哪执行按「工具」一节选择。

复现链一旦超过两三步，就在 Mac 的工作副本写成 `solve.py` 或 `solve.sh`，后面的汇报指这个文件。

远程只使用用户给出的那个地址和端口。先设超时。题目以外的主机不在范围内。

数字、偏移、密钥、地址、flag 只写工具输出或文件里实际出现的值。

完成

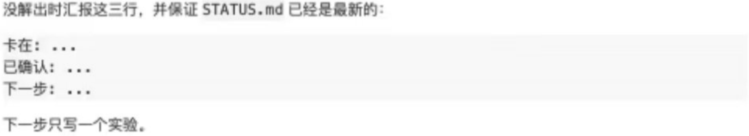
同时满足才算解出：

1. flag 字符串来自程序输出、远程返回或题面规定的结果。
2. 工作目录里有一条能再次得到它的命令或脚本。
3. 题面规定了格式时，flag 与格式一致。

向用户只汇报：

```
flag: ...
复现: ...
```

复现用工作目录里的命令或脚本路径。必要的原因用两句话写在后面。



没解出时汇报这三行，并保证 `STATUS.md` 已经是最新的：

```
卡在：...
已确认：...
下一步：...
```

下一步只写一个实验。

告诉它我的基础环境信息、工作目录（防止乱放文件）、工具信息（MCP、Ubuntu虚拟机）等。

并让他进行三次实验循环，**如果没有进展，立刻停机（很重要）**然后输出**状态卡（卡壳点、已解决、下一步）**。

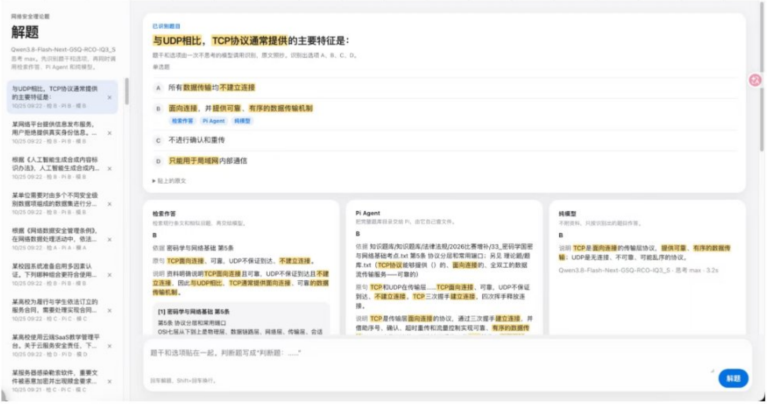
如果没有这段提示词，模型会无穷无尽的解下去，甚至走向错误的方向。而**停机+状态卡**可以更好的人机协同。

经过多次bench解题实测，发现这个是对模型解题能力最大的提升。本次比赛中的2道Web题目，我使用多并发去对黑盒题目进行信息收集，然后人工研判并将各自对话的有用信息汇总，交给一个新的对话重新探索。通过人机协同大幅度提高AI解题能力。

（Skills虽然冗余，但是这次比赛的密码题目：CRT泄露，纯LLM无法解决，我将Skills文件手动加到对话中，刚好弥补了模型的知识缺陷成功解出题目）

7.解题框架

理论题部分，我开发了一个**AI并行解题框架**：



网络安全理论题

解题

Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S 思考 max。先识别题干和选项，再同时调用检索作答、Pi Agent 和纯模型。

与UDP相比，TCP协议通常提供的主要特征是：
10/25 09:22 · 检 B · Pi B · 模 B

某网络平台提供信息发布服务，用户拒绝提供真实身份信息。...
10/25 09:22 · 检 B · Pi B · 模 B

根据《人工智能生成合成内容标识办法》，人工智能生成合成内...
10/25 09:22 · 检 B · Pi B · 模 B

某单位需要对由多个不同安全级别数据项组成的数据集进行分...
10/25 09:22 · 检 B · Pi B · 模 B

根据《网络数据安全管理条例》，在网络数据处理活动中，依法...
10/25 09:22 · 检 A · Pi A · 模 A

某校园系统准备启用多因素认证，下列哪种组合更符合使用...
10/25 09:22 · 检 B · Pi B · 模 B

某高校为履行与学生依法订立的服务合同，需要处理实现合同...
10/25 09:22 · 检 C · Pi C · 模 C

某高校使用云端SaaS教学管理平台，关于云服务安全责任，下...
10/25 09:22 · 检 D · Pi D · 模 D

某服务器感染勒索软件，重要文件被恶意加密并出现赎金要求...
10/25 09:21 · 检 C · Pi C · 模 C

已识别题目

与UDP相比，TCP协议通常提供的主要特征是：

题干和选项由一次不思考的模型调用识别，原文照抄。识别出选项 A、B、C、D。

单选题

A 所有数据传输均不建立连接

B 面向连接，并提供可靠、有序的数据传输机制

C 不进行确认和重传

D 只能用于局域网内部通信

▶ 贴上的原文

检索作答

检索现行条文和相似旧题，再交给模型。

B

依据 密码学与网络基础 第5条

原句 TCP面向连接、可靠，UDP不保证到达、不建立连接。

说明 资料明确说明TCP面向连接且可靠，UDP不保证到达且不建立连接，因此与UDP相比，TCP通常提供面向连接、可靠的数据传输机制。

[1] 密码学与网络基础 第5条

第5条 协议分层和常用端口

OSI七层从下到上是物理层、数据链路层、网络层、传输层、会话

Pi Agent

把完整题库目录交给 Pi，由它自己查文件。

B

依据 知识题库/知识题库/法律法规/2026比赛增补/33_密码学国密与网络基础考点.txt 第5条 协议分层和常用端口；另见 理论题/题库.txt（TCP协议能够提供（）的、面向连接的、全双工的数据流传输服务——可靠的）

原句 TCP和UDP在传输层……TCP面向连接、可靠，UDP不保证到达、不建立连接，TCP三次握手建立连接，四次挥手释放连接。

说明 TCP是传输层面向连接的协议，通过三次握手建立连接，并借助序号、确认、超时重传和流量控制实现可靠、有序的数据传

纯模型

不附资料，只按识别出的题目作答。

B

说明 TCP是面向连接的传输层协议，提供可靠、有序的数据传输；UDP是无连接、不可靠、可能乱序的协议。

Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S · 思考 max · 3.2s

题干和选项贴在一起。判断题写成“判断题：……”

回车解题，Shift+回车换行。

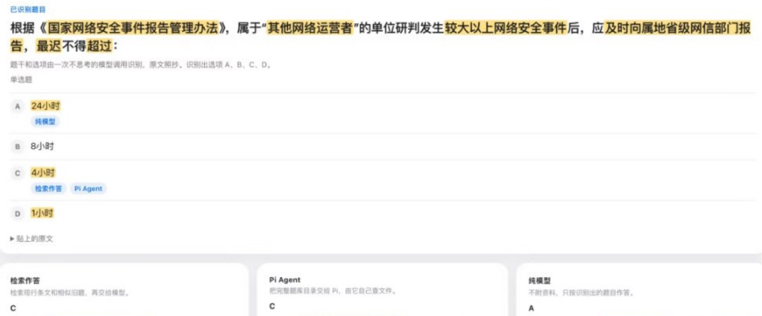
将选择题库**切片后建立 FTS5 全文索引**，检索时用 SQLite FTS5/BM25 做关键词匹配，再交给模型生成带引用的回答。

这个也是实测优化后的方法，RAG的向量检索会导致相似语义内容检索不准确。因此，我选择了关键词匹配方案。

当用户复制粘贴问答题时，系统会：

1. 调用小模型（无思考）将输入格式化，输出格式化后的 题目内容 与 选项。
2. 三路并行解题：
    - 知识库检索 + LLM API
    - Agent搜索知识库
    - 纯LLM API

通过三个结果交叉检验，得到最终答案。在本次比赛中，发现有1道题目是**纯LLM无法解决**的：



已识别题目

根据《国家网络安全事件报告管理办法》，属于“其他网络运营者”的单位研判发生较大以上网络安全事件后，应及时向属地省级网信部门报告，最迟不得超过：

题干和选项由一次不思考的模型调用识别，原文照抄。识别出选项 A、B、C、D。

单选题

A 24小时
纯模型

B 8小时

C 4小时
检索作答 Pi Agent

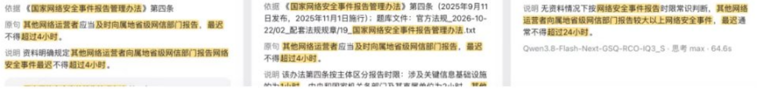
D 1小时

▸ 贴上的原文

检索作答
检索现行条文和相似旧题，再交给模型。
C

Pi Agent
把完整题库目录交给 Pi，由它自己查文件。
C

纯模型
不附资料，只按识别出的题目作答。
A



依据 《国家网络安全事件报告管理办法》第四条

原句 其他网络运营者应当及时向属地省级网信部门报告，最迟不得超过4小时。

说明 资料明确规定其他网络运营者向属地省级网信部门报告网络安全事件最迟不得超过4小时。

依据 《国家网络安全事件报告管理办法》第四条（2025年9月11日发布，2025年11月1日施行）；题库文件：官方法规_2026-10-22/02_配套法规规章/19_国家网络安全事件报告管理办法.txt

原句 其他网络运营者应当及时向属地省级网信部门报告，最迟不得超过4小时。

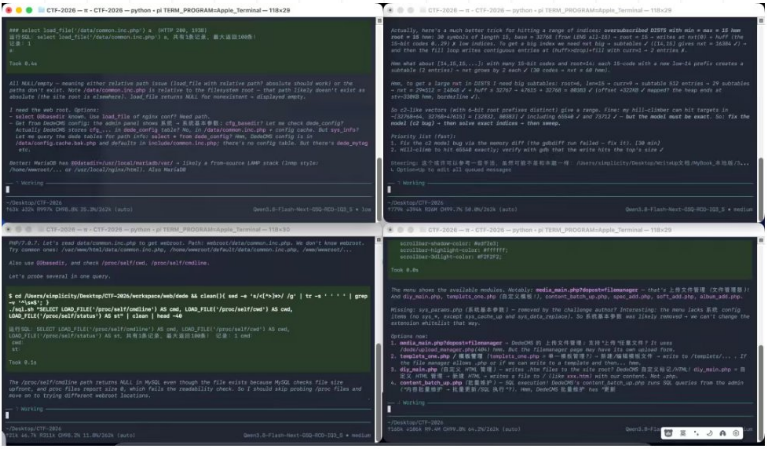
说明 该办法第四条按主体区分报告时限：涉及关键信息基础设施

说明 无资料情况下按网络安全事件报告时限常识判断，其他网络运营者向属地省级网信部门报告较大以上网络安全事件，最迟通常不得超过24小时。

Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S · 思考 max · 64.6s

这道题目模型出现知识缺失的问题，但知识库检索后丢给模型分析得到的答案是正确的。

使用 `Pi Agent` 作为解题的主力工具：



CTF-2026 — π - CTF-2026 — python ◂ pi TERM_PROGRAM=Apple_Terminal — 118×29

```
### select load_file('/data/common.inc.php') a  (HTTP 200, 1938)
运行SQL: select load_file('/data/common.inc.php') a, 共有1条记录, 最大返回100条!
记录: 1
a:

Took 0.4s

All NULL/empty — meaning either relative path issue (load_file with relative path? absolute should work) or the
paths don't exist. Note /data/common.inc.php is relative to the filesystem root — that path likely doesn't exist as
absolute (the site root is elsewhere). load_file returns NULL for nonexistent → displayed empty.

I need the web root. Options:
- select @@basedir known. Use load_file of nginx conf? Need path.
- Get from DedeCMS config: the admin panel shows 系统 → 系统基本参数: cfg_basedir? Let me check dede_config?
  Actually DedeCMS stores cfg_... in dede_config table? No, in /data/common.inc.php + config cache. But sys_info?
  Let me query the dede tables for path info: select * from dede_config? Hmm, DedeCMS config is in
  /data/config.cache.bak.php and defaults in include/common.inc.php; there's no config table. But there's dede_mytag
   etc.

Better: MariaDB has @@datadir=/usr/local/mariadb/var/ → likely a from-source LAMP stack (lnmp style:
/home/wwwroot/... or /usr/local/nginx/html). Also MariaDB

—— ⠋ Working ——————————————————————————————————————————————————————————
▌

~/Desktop/CTF-2026
↑63k ↓32k R997k CH96.8% 25.3%/262k (auto)                         Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S • low
```

CTF-2026 — π - CTF-2026 — python ◂ pi TERM_PROGRAM=Apple_Terminal — 118×29

```
Actually, here's a much better trick for hitting a range of indices: oversubscribed DISTS with min = max = 15 hmm
root = 15 hmm: 30 symbols of length 15, base = 32768 (from LENS all-15) → root = 15 → writes at nxt(0) + huff (the
15-bit codes 0..29) ✗ low indices. To get a big index we need nxt big → subtables ✓ ([14,15] gives nxt = 16384 ✓) →
and then the fill loop writes contiguous entries at (huff>>drop)+fill with curr=1 → 2 entries ✗.

Hmm what about [14,15,15,...]: with many 15-bit codes and root=14: each 15-code with a new low-14 prefix creates a
subtable (2 entries) → nxt grows by 2 each ✓ (30 codes → nxt ≤ 60 hmm).

Hmm, to get a large nxt in DISTS I need big subtables: root=6, len=15 → curr=9 → subtable 512 entries → 29 subtables
→ nxt = 29*512 = 14848 ✓ + huff ≤ 32767 → 47615 + 32768 = 80383 ✓ (offset +322KB ✓ mapped? the heap ends at
st+~330KB hmm, borderline ✓).

So c2-like vectors (with 6-bit root prefixes distinct) give a range. Fine: my hill-climber can hit targets in
~[32768+64, 32768+47615] = [32832, 80383] ✓ including 65540 ✓ and 73712 ✓ — but the model must be exact. So: fix the
model (c2 bug) → then solve exact indices → then sweep.

Priority list (fast):
1. Fix the c2 model bug via the memory diff (the gdbdiff run failed — fix it). [30 min]
2. Hill-climb to hit 65540 exactly; verify with gdb that the write hits the top's size ✓

Steering: 这个或许可以参考一些手法，虽然可能不是和本题一样：/Users/simplicity/Desktop/WriteUp文档/MyBook_本地版/3...
↳ Option+Up to edit all queued messages

—— ⠋ Working ——————————————————————————————————————————————————————————
▌

~/Desktop/CTF-2026
↑779k ↓394k R26M CH99.7% 50.0%/262k (auto)                     Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S • medium
```

CTF-2026 — π - CTF-2026 — python ◂ pi TERM_PROGRAM=Apple_Terminal — 118×30

```
PHP/7.0.7. Let's read data/common.inc.php to get webroot. Path: webroot/data/common.inc.php. We don't know webroot.
Try common ones: /var/www/html/data/common.inc.php, /home/wwwroot/default/data/common.inc.php, /www/wwwroot/...

Also use @@basedir, and check /proc/self/cwd, /proc/self/cmdline.

Let's probe several in one query.

$ cd /Users/simplicity/Desktop/CTF-2026/workspace/web/dede && clean(){ sed -e 's/<[^>]*>/ /g' | tr -s ' ' ' ' | grep
-v '^\s*$'; }
./sql.sh "SELECT LOAD_FILE('/proc/self/cmdline') AS cmd, LOAD_FILE('/proc/self/cwd') AS cwd,
LOAD_FILE('/proc/self/status') AS st" | clean | head -40

运行SQL: SELECT LOAD_FILE('/proc/self/cmdline') AS cmd, LOAD_FILE('/proc/self/cwd') AS cwd,
LOAD_FILE('/proc/self/status') AS st, 共有1条记录, 最大返回100条!  记录: 1 cmd:
 cwd:
 st:

Took 0.1s

The /proc/self/cmdline path returns NULL in MySQL even though the file exists because MySQL checks file size
upfront, and proc files report size 0, which fails the readability check. So I should skip probing /proc files and
move on to trying different webroot locations.

—— ⠋ Working ——————————————————————————————————————————————————————————
▌

~/Desktop/CTF-2026
↑21k ↓6.7k R311k CH96.2% 11.8%/262k (auto)                     Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S • medium
```

CTF-2026 — π - CTF-2026 — python ◂ pi TERM_PROGRAM=Apple_Terminal — 118×29

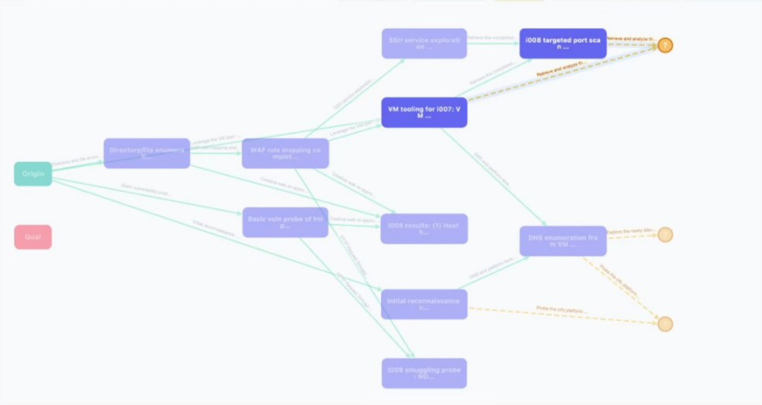
```
   scrollbar-shadow-color: #edf2e3;
   scrollbar-highlight-color: #ffffff;
   scrollbar-3dlight-color: #F2F2F2;

Took 0.0s

The menu shows the available modules. Notably: media_main.php?dopost=filemanager — that's 上传文件管理 (文件管理器)!
And diy_main.php, templets_one.php (自定义模板!), content_batch_up.php, spec_add.php, soft_add.php, album_add.php.

Missing: sys_params.php (系统基本参数) — removed by the challenge author? Interesting: the menu lacks 系统 config
items (no sys_*, except sys_cache_up and sys_data_replace). So 系统基本参数 was likely removed → we can't change the
extension whitelist that way.

Options now:
1. media_main.php?dopost=filemanager → DedeCMS 的 上传文件管理: 支持"上传"任意文件? It uses
   /dede/upload_manager.php(404) hmm. But the filemanager page may have its own upload form.
2. templets_one.php / 模板管理 (templets_one.php = 单一模板管理?) → 新建/编辑模板文件 → write to /templets/... . If
   the file manager allows .php or if we can write to a template and then... hmm.
3. diy_main.php (自定义 HTML 管理) — writes .htm files to the site root? DedeCMS 自定义标记/HTML! diy_main.php = 自
   定义 HTML 管理 → 新建 HTML → writes a file to / (like xxx.htm) with our content. Not .php.
4. content_batch_up.php (批量维护) — SQL execution! DedeCMS's content_batch_up.php runs SQL queries from the admin
   ("内容批量维护 → 批量更新/SQL 执行"?). Hmm, DedeCMS 批量维护 has "更新

—— ⠋ Working ——————————————————————————————————————————————————————————
▌

~/Desktop/CTF-2026
↑166k ↓186k R9.4M CH99.8% 64.2%/262k (auto)                     Qwen3.8-Fla
```

接入各种工具的MCP以及Ubuntu解题环境。

除此之外，我还用了 `Cairn` 框架

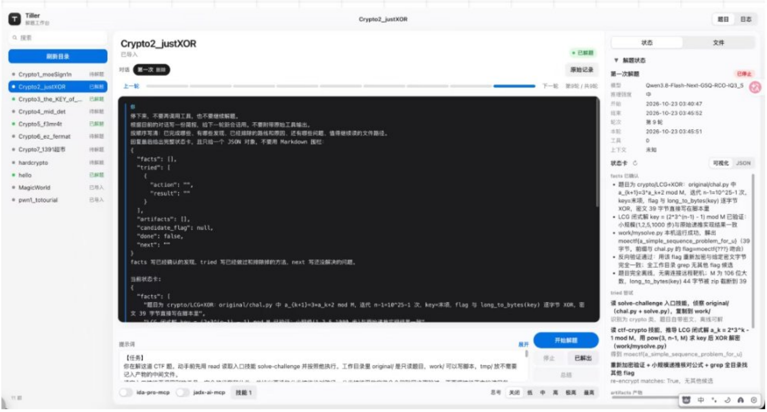
（https://github.com/oritera/Cairn）解Web题目。

之所以使用这个框架，是因为对于附件题目来说，本地的信息是足够的，主要考察模型的工具交互与推理能力。


Origin
Goal
Directory/file enumera ti...
WAF rule mapping co mplet...
Basic vuln probe of htt p...
SSH service explorati on ...
VM tooling for i007: V M ...
i008 results: (1) Host h...
Initial reconnaissance c...
i009 smuggling probe : NO...
i008 targeted port sca n ...
DNS enumeration fro m VM ...
?
Retrieve and analyze th...
Probe the city platform...

而Web题目是黑盒，需要多Agent并发去收集足够的信息，并整合信息后解题。

最后，还开发了一个 `CTF-Tiller` 作为并发解题的补充：



Crypto2_justXOR

```
你
停下来，不要再调用工具，也不要继续解题。
根据目前的对话写一份简报，给下一轮新会话用。不要附带原始工具输出。
按顺序写清：已完成哪些、有哪些发现、已经排除的路线和原因、还有哪些问题、值得继续读的文件路径。
回复最后给出完整状态卡，且只给一个 JSON 对象，不要用 Markdown 围栏：
{
  "facts": [],
  "tried": [
    {
      "action": "",
      "result": ""
    }
  ],
  "artifacts": [],
  "candidate_flag": null,
  "done": false,
  "next": ""
}
facts 写已经确认的发现，tried 写已经做过和排除掉的方法，next 写还没解决的问题。

当前状态卡：
{
  "facts": [
    "题目为 crypto/LCG+XOR: original/chal.py 中 a_{k+1}=3*a_k+2 mod M，迭代 n-1=10^25-1 次，key=末项，flag 与 long_to_bytes(key) 逐字节 XOR，密文 39 字节直接写在脚本里",
```

【任务】
你在解这道 CTF 题，动手前先用 read 读取入口技能 solve-challenge 并按照他执行。工作目录里 original/ 是只读题目，work/ 可以写脚本，tmp/ 放不需要记入产物的中间文件。

第一次解题

模型 Qwen3.8-Flash-Next-GSQ-RCO-IQ3_S
推理强度 中
开始 2026-10-23 03:40:47
结束 2026-10-23 03:45:52
轮次 第 9 轮
本轮 2026-10-23 03:45:51
工具 0
上下文 未知

状态卡

facts 已确认

- 题目为 crypto/LCG+XOR: original/chal.py 中 a_(k+1)=3*a_k+2 mod M，迭代 n-1=10^25-1 次，key=末项，flag 与 long_to_bytes(key) 逐字节 XOR，密文 39 字节直接写在脚本里
- LCG 闭式解 key = (2*3^(n-1) - 1) mod M 已验证：小规模(1,2,5,1000 步)与原始递推实现结果一致
- work/mysolve.py 本机运行成功，解出 moectf{a_simple_sequence_problem_for_u}（39 字节，前缀与 chal.py 的 flag=moectf{???} 吻合）
- 反向验证通过：用该 flag 重新加密与给定密文字节完全一致；全工作目录 grep 无其他 flag 候选
- 题目完全离线，无需连接远程靶机；M 为 106 位大数，long_to_bytes(key) 44 字节被 zip 截断到 39

tried 尝试

读 solve-challenge 入口技能，侦察 original/（chal.py + solve.py），复制到 work/
识别为 crypto 类，题目自带密文，离线可解

读 ctf-crypto 技能，推导 LCG 闭式解 a_k = 2*3^k - 1 mod M，用 pow(3, n-1, M) 求 key 后 XOR 解密（work/mysolve.py）
得到 moectf{a_simple_sequence_problem_for_u}

重新加密验证 + 小规模递推核对公式 + grep 全目录找其他 flag
re-encrypt matches: True，无其他候选

artifacts 产物

可以更直观的看到每个题目的结题进度，并回退到某一步的对话，修改`状态卡`重新解题。

并且便捷的接入了各种`mcp`和`skills`的开关选项，相比于`pi Agent`更方便的操作和处理并发。

相关工具后续会陆续开源（开发时间仓促，很多配置与本机绑定，需要逐步优化）。

8.拓展阅读

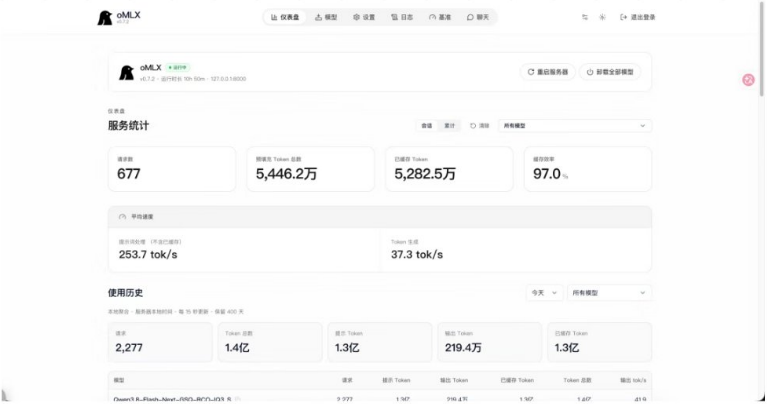
Strata（https://github.com/Niko1221/Strata）：

Moe模型优化，可以让小显存跑大模型。原理大概为**显存放常用专家缓存、内存放全部专家、SSD放n-gram查找表**。

Splash（https://github.com/incoai/splash）：针对Mac Silicon架构和模型进行专用优化，让Qwen跑的更快。

Agent使用日志

比赛当天oMLX的使用日志（共2270次请求，输出2193k Token）：



oMLX
v0.7.2 · 运行时长 10h 50m · 127.0.0.1:8000

仪表盘

服务统计

请求数
677

预填充 Token 总数
5,446.2万

已缓存 Token
5,282.5万

缓存效率
97.0 %

平均速度

提示词处理（不含已缓存）
253.7 tok/s

Token 生成
37.3 tok/s

使用历史

本地聚合 · 服务器本地时间 · 每 15 秒更新 · 保留 400 天

请求
2,277

Token 总数
1.4亿

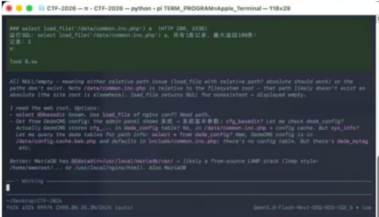
提示 Token
1.3亿

输出 Token
219.4万

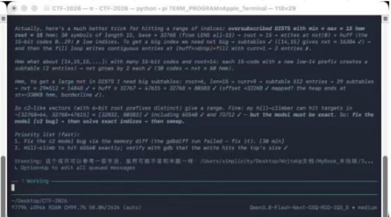
已缓存 Token
1.3亿

模型	请求	提示 Token	输出 Token	已缓存 Token	Token 总数	输出 tok/s
Qwen3.8-Flash-Next-GSQ-RCQ-IQ3 S	2,277	1.3亿	219.4万	1.3亿	1.4亿	41.9

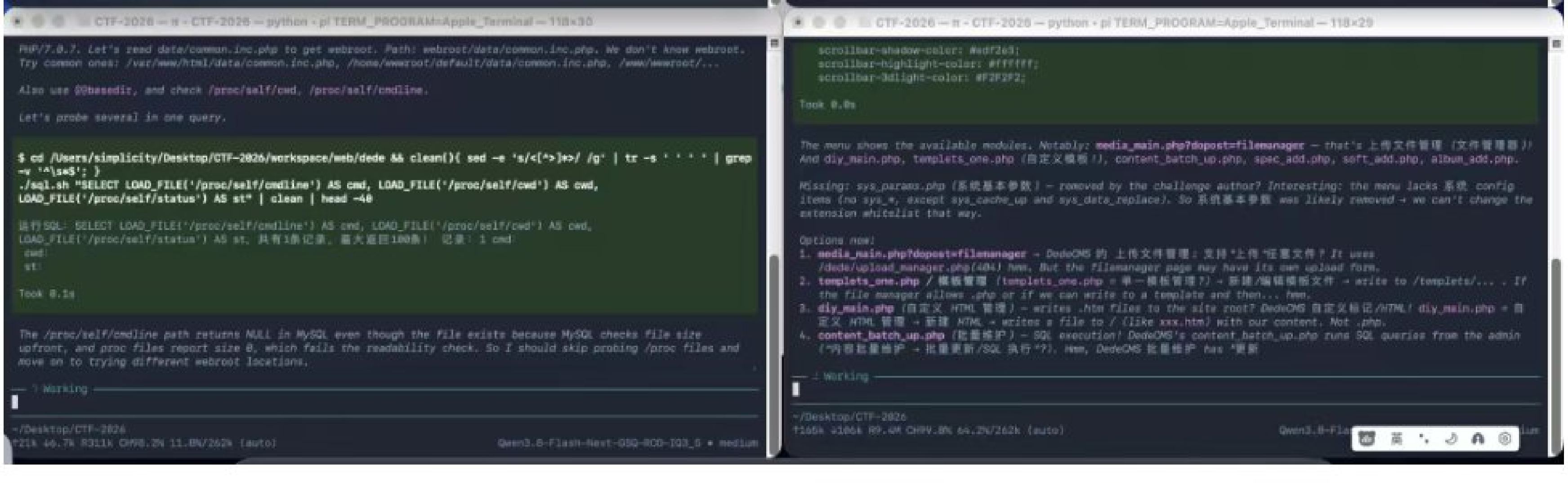
Pi Agent四并发解题：



```
CTF-2026 — π - CTF-2026 — python • pi TERM_PROGRAM=Apple_Terminal — 118×29

### select load_file('/data/common.inc.php') a  (HTTP 200, 1938)
运行SQL: select load_file('/data/common.inc.php') a, 共有1条记录, 最大返回100条!
记录: 1
a:

Took 0.4s

All NULL/empty — meaning either relative path issue (load_file with relative path? absolute should work) or the
paths don't exist. Note /data/common.inc.php is relative to the filesystem root — that path likely doesn't exist as
absolute (the site root is elsewhere). load_file returns NULL for nonexistent → displayed empty.

I need the web root. Options:
- select @@basedir known. Use load_file of nginx conf? Need path.
- Get from DedeCMS config: the admin panel shows 系统 → 系统基本参数: cfg_basedir? Let me check dede_config?
  Actually DedeCMS stores cfg_... in dede_config table? No, in /data/common.inc.php + config cache. But sys_info?
  Let me query the dede tables for path info: select * from dede_config? Hmm, DedeCMS config is in
  /data/config.cache.bak.php and defaults in include/common.inc.php; there's no config table. But there's dede_mytag
   etc.

Better: MariaDB has @@datadir=/usr/local/mariadb/var/ → likely a from-source LAMP stack (lnmp style:
/home/wwwroot/... or /usr/local/nginx/html). Also MariaDB

— ⠋ Working ————————————————————————————————————————————————
█

~/Desktop/CTF-2026
↑63k ↓32k R997k CH98.8% 35.3%/262k (auto)                        Qwen3.6-Flash-Next-OSQ-RCO-IQ3_S • low
```



```
CTF-2026 — π - CTF-2026 — python • pi TERM_PROGRAM=Apple_Terminal — 118×29

Actually, here's a much better trick for hitting a range of indices: oversubscribed DISTS with min = max = 15 hmm
root = 15 hmm: 30 symbols of length 15, base = 32768 (from LENS all-15) → root = 15 → writes at nxt(0) + huff (the
15-bit codes 0..29) ✗ low indices. To get a big index we need nxt big → subtables ✓ ([14,15] gives nxt = 16384 ✓) →
and then the fill loop writes contiguous entries at (huff>>drop)+fill with curr=1 → 2 entries ✗.

Hmm what about [14,15,15,...]: with many 15-bit codes and root=14: each 15-code with a new low-14 prefix creates a
subtable (2 entries) → nxt grows by 2 each ✓ (30 codes → nxt ≈ 60 hmm).

Hmm, to get a large nxt in DISTS I need big subtables: root=6, len=15 → curr=9 → subtable 512 entries → 29 subtables
→ nxt = 29*512 = 14848 ✓ + huff ≤ 32767 → 47615 + 32768 = 80383 ✓ (offset +322KB ✓ mapped? the heap ends at
st+~338KB hmm, borderline ✓).

So c2-like vectors (with 6-bit root prefixes distinct) give a range. Fine: my hill-climber can hit targets in
~[32768+64, 32768+47615] = [32832, 80383] ✓ including 65540 ✓ and 73712 ✓ — but the model must be exact. So: fix the
model (c2 bug) → then solve exact indices → then sweep.

Priority list (fast):
1. Fix the c2 model bug via the memory diff (the gdbdiff run failed — fix it). [30 min]
2. Hill-climb to hit 65540 exactly; verify with gdb that the write hits the top's size ✓

Steering: 这个题你可以参考一些手法，虽然可能不是和本题一样: /Users/simplicity/Desktop/WriteUp文档/MyBook_本地版/S...
↳ Option+Up to edit all queued messages

— ⠋ Working ————————————————————————————————————————————————
█

~/Desktop/CTF-2026
↑779k ↓394k R26M CH99.7% 50.0%/262k (auto)                       Qwen3.6-Flash-Next-OSQ-RCO-IQ3_S • medium
```



```
CTF-2026 — π · CTF-2026 — python · pi TERM_PROGRAM=Apple_Terminal — 118×30
PHP/7.0.7. Let's read data/common.inc.php to get webroot. Path: webroot/data/common.inc.php. We don't know webroot.
Try common ones: /var/www/html/data/common.inc.php, /home/wwwroot/default/data/common.inc.php, /www/wwwroot/...

Also use @@basedir, and check /proc/self/cwd, /proc/self/cmdline.

Let's probe several in one query.

$ cd /Users/simplicity/Desktop/CTF-2026/workspace/web/dede && clean(){ sed -e 's/<[^>]*>/ /g' | tr -s ' ' ' ' | grep
-v '^\s*$'; }
./sql.sh "SELECT LOAD_FILE('/proc/self/cmdline') AS cmd, LOAD_FILE('/proc/self/cwd') AS cwd,
LOAD_FILE('/proc/self/status') AS st" | clean | head -40

运行SQL: SELECT LOAD_FILE('/proc/self/cmdline') AS cmd, LOAD_FILE('/proc/self/cwd') AS cwd,
LOAD_FILE('/proc/self/status') AS st, 共有1条记录, 最大返回100条!  记录: 1 cmd
 cwd:
 st:

Took 0.1s

The /proc/self/cmdline path returns NULL in MySQL even though the file exists because MySQL checks file size
upfront, and proc files report size 0, which fails the readability check. So I should skip probing /proc files and
move on to trying different webroot locations.

— ⁝ Working ————————————————————————————————————
▌
~/Desktop/CTF-2026
↑21k ↓6.7k R311k CH98.2% 11.8%/262k (auto)                    Qwen3.6-Flash-Next-GSQ-RCD-IQ3_S • medium
```

```
CTF-2026 — π · CTF-2026 — python · pi TERM_PROGRAM=Apple_Terminal — 118×29
   scrollbar-shadow-color: #edf2e3;
   scrollbar-highlight-color: #ffffff;
   scrollbar-3dlight-color: #F2F2F2;

Took 0.0s

The menu shows the available modules. Notably: media_main.php?dopost=filemanager — that's 上传文件管理 (文件管理器)!
And diy_main.php, templets_one.php (自定义模板!), content_batch_up.php, spec_add.php, soft_add.php, album_add.php.

Missing: sys_params.php (系统基本参数) — removed by the challenge author? Interesting: the menu lacks 系统 config
items (no sys_*, except sys_cache_up and sys_data_replace). So 系统基本参数 was likely removed → we can't change the
extension whitelist that way.

Options now:
1. media_main.php?dopost=filemanager → DedeCMS 的 上传文件管理: 支持"上传"任意文件? It uses
   /dede/upload_manager.php(404) hmm. But the filemanager page may have its own upload form.
2. templets_one.php / 模板管理 (templets_one.php = 单一模板管理?) → 新建/编辑模板文件 → write to /templets/... . If
   the file manager allows .php or if we can write to a template and then... hmm.
3. diy_main.php (自定义 HTML 管理) — writes .htm files to the site root? DedeCMS 自定义标记/HTML! diy_main.php = 自
   定义 HTML 管理 → 新建 HTML → writes a file to / (like xxx.htm) with our content. Not .php.
4. content_batch_up.php (批量维护) — SQL execution! DedeCMS's content_batch_up.php runs SQL queries from the admin
   ("内容批量维护 → 批量更新/SQL 执行"?). Hmm, DedeCMS 批量维护 has "更新

— ⁝ Working ————————————————————————————————————
▌
~/Desktop/CTF-2026
↑165k ↓106k R9.4M CH99.8% 64.2%/262k (auto)                    Qwen3.6-Fla
```

WriteUp

AI Agent时代应该没有人去看古法题解了吧（bushi

搬运下星宇Sec师傅的文章：

“瀚海数据库杯”第十三届山东省大学生网络安全技能大赛题目WP

授之以鱼不如授之以渔，本文主要分享LLM部署相关知识以及AI协同解决方案。

附件已经提供给各位师傅了，师傅们可以下载后让Agent做题并输出WriteUp。

这里分享下1解Web题 `NodeScope（不太简单的web 5）` 的攻击链：

```
tool.php.bak 泄露源码
→ login.php 子串密码拓展字符绕过 → admin/hacked
→ tool.php @unserialize → POP A::__destruct
→ 黑名单绕过 http://0x7f000001/… + gopher://0x
→ 仅 127.0.0.1 可访问的 admin.php / doLogin.ph
→ DOMDocument::loadXML(..., LIBXML_NOENT|LIB
→ php://filter 读 /flag 外带
```

喜欢作者

“ 请作者喝杯咖啡 ”

技术杂谈 · 目录 ≡

‹ 上一篇 · 2分钟速览 | ChatGPT Images 2.0 | 完爆Nano ...

阅读 2.2万　　　　不喜欢

广告

让多模态 AI 真正跑在 端侧
端侧原生
流式多模态
实时 · 多模态 · 原生 · 端侧
端侧部署
数据不出机
多模态输入
图像 / 语音 / 文本 / 视频
流式交互
实时响应

开发者社区系列课程，三期讲清物理AI的端侧原生

点击了解更多 ›

开发者社区系列课程，三期讲清物理AI的端侧原生

VLX端侧流式多模态模型

关注公众号

留言 103

写留言

Real返璞归真 作者 昨天　　回复 5

无偿开源本地CTF大模型解决方案也是打破信息差，希望为推动赛制改革贡献一份力量

置顶 作者赞过

Gao 河北 2天前　　回复 8

1.ai的写代码能力已经很强（共识），同时漏洞作为coding中的特殊产物，ai挖洞编排攻击能力已经很强了（参考artex的棒子银行事件）
2.ctf本身就是挖洞、渗透的副产品，通过这种比赛检验一下攻击能力，ai做这些更是简单（毕竟先进模型都已经解数学界的难题了）
3.在上述基础上，ctf比赛不让使用ai已经成为了无法实现的硬约束，也不应该不让使用，反而应该迎合这个趋势。比如：赛事方允许手搓ctf，允许ai自动化。最终以解题速度、准确率以及token消耗率作为参考纬度设置不同权重，最终决策答题排名。
4.如果还在硬性约束不让用ai答题，仿佛就是在做一场工具速度比拼大赛：外面已经上蒸汽机、电动汽车开始跑了，自己还在比拼谁养的马更好跑的更快，完全脱离了发展轨道。
以上：个人理解和判断